

CHAPTER 11

11. NestJS + TypeORM — 인메모리를 진짜 DB로

Node.js · Express · NestJS 강의 · 2026

10장에서 만든 NestJS 서버는 잘 돌아갔다. 하지만 한 가지 결정적인 한계가 있었다. 데이터가 배열 하나에 담겨 있어서, 서버를 끄면 전부 사라진다. `users` 배열에 사용자를 백 명 넣어 뒀어도 `Ctrl+C` 한 번이면 빈 배열로 돌아간다. 학습용으로는 충분했지만 실제 서비스는 이렇게 못 만든다.

이 장에서는 그 배열을 **PostgreSQL**로 바꾼다. 컨트롤러도, DTO도, 엔드포인트도 그대로다. 바뀌는 건 오직 데이터를 어디에 어떻게 저장하느냐 — 서비스 계층 한 곳뿐이다. 이 교체에 쓰는 도구가 **TypeORM**이다.

데이터베이스에 직접 SQL을 쓰는 대신, 자바스크립트 클래스로 테이블을 정의하고 메서드 호출로 질의한다. 이렇게 객체 (Object)와 관계형 DB(Relational)를 잇는 것을 **ORM**이라 부른다. 같은 도메인을 12장에서는 또 다른 ORM인 Prisma로 다시 만든다. 두 장을 비교하면 DB 계층만 바뀌 끼우는 구조가 또렷해진다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- `@Entity` 클래스로 테이블 스키마를 코드로 정의하는 법
- `TypeOrmModule.forRoot` (DB 연결)와 `forFeature` (모듈별 리포지토리 등록)의 역할 분담
- `@InjectRepository` 로 `Repository<T>` 를 주입받아 CRUD를 메서드로 처리하는 법
- `@ManyToOne` / `@OneToMany` 로 테이블 사이의 관계를 맺고, `relations` 옵션으로 연관 데이터를 불러오는 법
- `synchronize: true` 가 무엇을 자동으로 처리하는지, 그리고 왜 실무에서는 끄는지

10장과 같은 도메인을 그대로 두고, 데이터를 저장하는 계층만 바꾼다. 달라진 부분에 초점을 두면 된다.

주제	핵심
부트스트랩	<code>.env</code> 로드, 전역 검증, 포트 3001
DB 연결	<code>forRoot</code> 로 연결과 엔티티를 한 번 등록
엔티티	클래스가 곧 테이블 스키마
관계	<code>@ManyToOne</code> 으로 작성자 관계 정의
리포지토리 등록	<code>forFeature</code> 로 모듈별 리포지토리
CRUD 서비스	배열 대신 리포지토리로 처리
연관 로드	<code>relations</code> 로 작성자 함께 불러오기
HTTP 계층	10장과 동일 — 컨트롤러·DTO는 안 바뀐다

핵심 개념

ORM — 테이블을 클래스로 다룬다

관계형 데이터베이스의 데이터는 테이블에 들어 있다. 테이블은 행과 열로 된 격자이고, 거기에 값을 넣고 빼려면 보통 SQL 을 쓴다. `INSERT INTO users..` , `SELECT * FROM users WHERE id = 1` 같은 문장이다.

ORM(Object-Relational Mapping)은 이 SQL을 객체와 메서드로 바꾸는 도구다. 테이블 하나를 클래스 하나로, 행 하나를 그 클래스의 인스턴스 하나로 대응시킨다. 그러면 `SELECT` 는 `repository.find()` 가 되고, `INSERT` 는 `repository.save()` 가 된다. SQL 문자열을 손으로 짜는 대신, 타입이 붙은 자바스크립트 코드로 DB를 다룬다.

TypeORM은 Node 진영에서 가장 널리 쓰이는 ORM 중 하나이고, NestJS와 궁합이 좋다. NestJS의 의존성 주입과 자연스럽게 맞물린다.

엔티티 — 클래스가 곧 스키마다

TypeORM에서 테이블의 모양은 별도의 SQL 파일이 아니라 클래스 정의가 결정한다. 클래스에 `@Entity()` 를 붙이고, 각 속성에 `@Column()` 같은 데코레이터를 붙이면, 그 클래스 자체가 테이블의 설계도가 된다.

```
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true })
  email: string;
}
```

이 한 덩어리가 "id(자동 증가 기본키)와 email(유니크) 열을 가진 users 테이블"을 정의한다. 스키마와 코드가 한 파일에 있으니, 모델을 고치면 테이블도 같이 바뀐다. 이 자동 반영을 맡는 게 뒤에 나올 `synchronize` 다.

리포지토리 패턴 — 저장소를 객체로 추상화한다

엔티티 하나마다 리포지토리(Repository)가 따라온다. 리포지토리는 "그 테이블에 대한 저장·조화·삭제 창구"다.

`Repository<User>` 는 User 테이블만 다루는 전담 객체이고, `find()` , `findOne()` , `save()` , `delete()` 같은 메서드를 갖췄다.

서비스는 이 리포지토리를 주입받아 쓴다. 데이터가 메모리 배열이든 PostgreSQL이든, 서비스 입장에서는 리포지토리에 요청하는 형태가 동일하다. 그래서 저장소를 통째로 바꿔도 서비스 코드의 모양이 크게 흔들리지 않는다. 10장의

`this.users.find(. )` 가 11장에서 `this.usersRepository.findOne(. )` 으로 바뀔 뿐이다.

콜아웃 활용 — 코드 안의 주석을 길잡이로

예제 코드에는 핵심 줄을 가리키는 표식 주석이 군데군데 박혀 있다. 본문 해설과 함께 그 주석을 같이 읽으면, "이 줄이 왜 중요한지"가 코드 안에서 한 번 더 짚인다. 해설서를 덮고 코드만 다시 볼 때도 이 표식이 단서가 된다.

준비

먼저 이 장에서 쓸 빈 데이터베이스를 하나 만든다. 로컬에 PostgreSQL이 설치돼 있어야 한다. 테이블은 앱이 알아서 만들 테니, 빈 DB만 준비하면 된다.

```
createdb nest_typeorm
# 또는: psql -c "CREATE DATABASE nest_typeorm;"
```

다음으로 의존성을 설치하고, 접속 정보를 담은 `.env` 를 만든다. `.env.example` 을 복사해 본인 PostgreSQL 계정으로 채운다.

```
npm install
cp .env.example .env # DB_USER / DB_PASSWORD 를 본인 계정으로 수정
npm run start:dev    # → [TypeORM] http://localhost:3001
```

`.env.example` 의 내용은 이렇다. `DB_USER` 와 `DB_PASSWORD` 두 줄을 자기 계정으로 바꾸는 게 핵심이다.

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=USER      # ← 본인 PostgreSQL 사용자명
DB_PASSWORD=PASSWORD # ← 비밀번호 (없으면 빈 값)
DB_NAME=nest_typeorm
```

서버는 **3001** 포트에서 뜬다. 10장이 3000을 썼으니, 두 서버를 동시에 띄워 비교해도 포트가 안 부딪친다. 첫 실행 때 콘솔에 `CREATE TABLE` 같은 SQL이 주르륵 찍히면, 엔티티 기준으로 테이블이 자동 생성되고 있다고 보면 된다.

부트스트랩 — 환경 변수와 전역 검증

```
import "dotenv/config"; // .env → process.env (다른 import보다 먼저 실행)
import { NestFactory } from "@nestjs/core";
import { ValidationPipe } from "@nestjs/common";
import { AppModule } from "../app.module";

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // DTO 유효성 검사를 전역으로 적용 (Prisma 앱과 동일하게)
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));

  await app.listen(3001);
  console.log("[TypeORM] http://localhost:3001 (users, posts)");
}
bootstrap();
```

10장의 부트스트랩과 거의 같지만, 맨 윗줄이 새로 들어왔다.

- `import "dotenv/config"` — 이 줄이 **가장 먼저** 와야 한다. `.env` 파일을 읽어 `process.env` 에 값을 올린다. 뒤에 나올 루트 모듈이 `process.env.DB_USER` 같은 값을 읽어 DB에 접속하는데, 그 모듈이 로드되기 전에 환경 변수가 준비돼 있어야 한다. import 순서가 곧 실행 순서이므로, 이 줄을 다른 import보다 위에 둔다.
- `app.useGlobalPipes(new ValidationPipe(.))` — DTO 검증을 앱 전체에 건다. `whitelist: true` 는 DTO에 없는 필드를 들어온 본문에서 잘라내고, `transform: true` 는 받은 값을 DTO 타입에 맞춰 변환한다. 이 한 줄로 모든 컨트롤러의 `@Body()` 가 자동으로 검증을 거친다.
- `await app.listen(3001)` — 3001 포트에서 요청을 받기 시작한다.

검증과 부트스트랩 방식은 10장과 똑같다. DB를 붙였다고 HTTP 입구까지 달라지지는 않는다.

DB 연결 — 루트 모듈에 한 번 심는다

여기가 이 장에서 진짜 새로운 첫 지점이다. 앱의 루트 모듈에 DB 연결 설정을 한 번 심는다.

```
import { Module } from "@nestjs/common";
import { TypeOrmModule } from "@nestjs/typeorm";
import { UsersModule } from "../users/users.module";
import { PostsModule } from "../posts/posts.module";
import { User } from "../users/user.entity";
import { Post } from "../posts/post.entity";

@Module({
  imports: [
    // ▼▼▼ TypeORM 핵심: 루트에서 DB 연결을 한 번 설정하고, 엔티티 목록을 등록한다 ▼▼▼
    TypeOrmModule.forRoot({
      type: "postgres",
      host: process.env.DB_HOST ?? "localhost",
      port: Number(process.env.DB_PORT ?? 5432),
      username: process.env.DB_USER ?? "myuser",
      password: process.env.DB_PASSWORD ?? "mypassword",
      database: process.env.DB_NAME ?? "nest_typeorm",
      entities: [User, Post],
      synchronize: true, // 개발 전용: 엔티티 클래스 기준으로 테이블을 자동 생성/변경
      logging: true, // 실행되는 SQL을 콘솔에 찍어 학습에 도움
    }),
    UsersModule,
    PostsModule,
  ],
})
export class AppModule {}
```

`TypeOrmModule.forRoot()` 가 이 설정의 전부다. 앱 전체에서 **딱 한 번** 호출하는 DB 연결 설정이다. 옵션을 줄별로 보자.

- `type: "postgres"` — 어떤 데이터베이스를 쓸지. MySQL이면 `"mysql"` 로 바꾸면 된다.
- `host ~ database` — 접속 정보다. 모두 `process.env` 에서 읽고, 값이 없으면 ?? 뒤의 기본값을 쓴다. `.env` 를 제대로 채웠다면 기본값은 쓰이지 않는다. `Number(process.env.DB_PORT ?? 5432)` — 환경 변수는 항상 문자열이므로, 포트는 숫자로 변환해 넘긴다.
- `entities: [User, Post]` — 이 연결이 관리할 엔티티 목록이다. 여기에 등록된 클래스들이 테이블 후보가 된다.
- `synchronize: true` — 엔티티 클래스를 보고 테이블을 자동으로 만들고 맞춘다. 첫 실행 때 `users-posts` 테이블이 없으면 만들고, 엔티티에 열을 추가하면 다음 실행 때 테이블에도 그 열이 붙는다. 마이그레이션 명령을 따로 돌릴 필요가 없어 학습에 편하다. 다만 운영에서는 위험하다(아래 박스 참고).
- `logging: true` — 실행되는 SQL을 콘솔에 그대로 찍는다. `find()` 한 번이 어떤 `SELECT` 로 번역되는지 눈으로 볼 수 있어, ORM이 뒤에서 무슨 일을 하는지 학습하기 좋다.

`forRoot` 아래에 `UsersModule`, `PostsModule` 을 같이 import한다. DB 연결은 루트에 한 번, 도메인 모듈은 그 옆에 나란히 — 이게 NestJS + TypeORM 앱의 기본 골격이다.

`synchronize: true` 를 운영에서 쓰지 않는 이유 — 이 옵션은 엔티티와 테이블을 강제로 일치시킨다. 편하지만, 엔티티에서 열을 지우면 테이블의 그 열도 데이터째 날아갈 수 있다. 개발 중 스키마가 자주 바뀔 때만 켜고, 운영에

서는 곧 뒤 **마이그레이션**으로 스키마 변경을 한 단계씩 통제한다. 다음 절에서 이어 설명한다.

엔티티 — 클래스가 곧 테이블이다

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  OneToMany,
  CreateDateColumn,
} from "typeorm";
import { Post } from "../posts/post.entity";

// TypeORM에서는 "클래스 + 데코레이터"가 곧 테이블 스키마다.
// 이 클래스 정의가 synchronize:true에 의해 실제 테이블로 만들어진다.
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true })
  email: string;

  @Column()
  name: string;

  // 1:N 관계 - 사용자 한 명이 여러 게시글을 가진다.
  @OneToMany(() => Post, (post) => post.author)
  posts: Post[];

  @CreateDateColumn()
  createdAt: Date;
}
```

10장에서는 사용자를 `interface User` 로 정의했다. 단순한 타입 표시일 뿐, 실행 시점에는 아무것도 아니었다. 11장의 엔티티는 다르다. 데코레이터가 붙은 **실재하는 클래스**이고, 이 정의가 그대로 테이블 설계도가 된다. 줄별로 어떤 열이 만들어지는지 보자.

- `@Entity()` — "이 클래스를 테이블로 다뤄라"는 표시다. 클래스 이름 `User` 를 따라 기본적으로 `user` 테이블이 생긴다.
- `@PrimaryGeneratedColumn()` `id` — **자동 증가 기본키**다. 새 행을 넣을 때마다 DB가 1, 2, 3...을 알아서 매긴다. 10장의 `nextId++` 를 손으로 굴리던 일을 DB가 대신한다.
- `@Column({ unique: true })` `email` — 일반 열인데 `unique` 제약이 붙었다. 같은 이메일을 두 번 넣으려 하면 DB가 거부한다. 10장에서 `users.some(. )` 으로 직접 검사하던 중복 방지를, 여기서는 DB 제약으로 못 박는다.

- `@Column()` `name` — 평범한 문자열 열이다.
- `@OneToMany(() => Post, (post) => post.author)` `posts` — 관계 열이다. "사용자 한 명이 게시글 여럿을 가진다"는 1:N 관계를 표현한다. 주의할 점은, 이 열은 `users` 테이블에 실제 컬럼으로 생기지 않는다. 관계의 실제 외래 키는 `posts` 쪽(`authorId`)에 있고, 여기 `posts` 는 "그 관계를 거꾸로 따라가 내 글들을 모아 오는" 가상의 통로다. 두 번째 인자 `(post) => post.author` 가 "Post의 author 필드로 연결돼 있다"는 짝을 알려 준다.
- `@CreateDateColumn()` `createdAt` — 행이 생성된 시각을 DB가 자동으로 채운다. 10장에서 `new Date()` 를 직접 넣던 것과 같은 결과를, 데코레이터 하나로 얻는다.

이렇게 정의한 클래스가 `synchronize: true` 를 만나 진짜 `user` 테이블로 만들어진다. 코드를 고치면 스키마가 따라온다.

관계 — 외래 키를 든 주인 쪽

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  ManyToOne,
  JoinColumn,
  CreateDateColumn,
} from "typeorm";
import { User } from "../users/user.entity";

@Entity()
export class Post {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @Column("text")
  content: string;

  // N:1 관계 - 게시글은 한 명의 작성자에 속한다.
  // onDelete: "CASCADE" - 작성자가 삭제되면 그 글도 함께 삭제.
  @ManyToOne(() => User, (user) => user.posts, { onDelete: "CASCADE" })
  @JoinColumn({ name: "authorId" })
  author: User;

  // FK 컬럼을 직접 노출 (author 객체 없이 authorId만으로 생성 가능하게)
  @Column()
  authorId: number;

  @CreateDateColumn()
  createdAt: Date;
}
```

게시글 엔티티에서 새로 볼 건 `@Column("text")` 와 관계 정의 세 줄이다.

- `@Column("text") content` — 본문은 길어질 수 있으니, 길이 제한이 있는 일반 문자열 대신 `text` 타입으로 지정한다. 괄호 안에 DB 컬럼 타입을 직접 넘기는 형태다.
- `@ManyToOne(() => User, (user) => user.posts, { onDelete: "CASCADE" }) author` — 관계의 주인 쪽이다. "게시글 여럿이 사용자 한 명에 속한다"는 N:1 관계다. User 엔티티의 `@OneToMany` 와 짝을 이루고, 이쪽이 실제 외래 키를 들고 있다. `onDelete: "CASCADE"` 는 "작성자가 삭제되면 그 사람의 글도 같이 지운다"는 DB 차원의 규칙이다. 글 없는 작성자, 작성자 없는 글이 남지 않게 한다.
- `@JoinColumn({ name: "authorId" })` — 위 관계가 테이블에 만들 외래 키 컬럼의 이름을 `authorId` 로 못 박는다. 이걸 지정하지 않으면 TypeORM이 임의의 이름을 만든다.

- `@Column()` `authorId` — 외래 키 값을 직접 다룰 수 있게 노출한 열이다. 이게 있어서 게시글을 만들 때 `author` 객체 전체를 채워 넘기지 않고, `{..., authorId: 1}` 처럼 숫자 하나만 줘도 된다. DTO가 `authorId: number` 만 받는 것과 정확히 맞물린다.

여기서 관계의 양쪽 그림이 완성된다. User에는 `@OneToMany posts` (거꾸로 따라가는 통로), Post에는 `@ManyToOne author` + `authorId` (실제 외래 키). 같은 관계를 두 엔티티가 각자의 방향에서 바라본다.

```

User (1) —< @OneToMany posts          Post (N) —> @ManyToOne author
  id  ←────────────────────────── authorId (FK)

```

리포지토리 등록 — 모듈마다 `forFeature`

```

import { Module } from "@nestjs/common";
import { TypeOrmModule } from "@nestjs/typeorm";
import { User } from "../user.entity";
import { UsersService } from "../users.service";
import { UsersController } from "../users.controller";

@Module({
  // forFeature: 이 모듈 안에서 User 리포지토리를 쓰겠다고 등록.
  imports: [TypeOrmModule.forFeature([User])],
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}

```

루트의 `forRoot` 가 DB 연결을 한 번 세웠다면, 각 도메인 모듈의 `forFeature` 는 "이 모듈에서는 이 엔티티의 리포지토리를 쓴다"고 **지역적으로** 등록한다.

- `TypeOrmModule.forFeature([User])` — 이 줄이 있어야 서비스에서 `@InjectRepository(User)` 로 `Repository<User>` 를 주입받을 수 있다. 등록하지 않고 주입을 시도하면 NestJS가 "그런 리포지토리를 모른다"며 부팅을 멈춘다.
- 나머지(`controllers` , `providers`)는 10장의 모듈과 똑같다.

역할을 나누면 이렇다. `forRoot` 는 "어느 DB에 어떻게 연결하나"(전역, 한 번), `forFeature` 는 "이 모듈에서 어떤 테이블을 다루나"(모듈마다). 게시글 모듈도 정확히 같은 모양으로 `forFeature([Post])` 를 등록한다.

CRUD 서비스 — 배열이 리포지토리로

이 장의 진짜 변화가 모인 자리다. 10장에서 `private users: User[] = []` 였던 자리가, 주입받은 리포지토리로 바뀐다.

```
import { Injectable, NotFoundException } from "@nestjs/common";
import { InjectRepository } from "@nestjs/typeorm";
import { Repository } from "typeorm";
import { User } from "../user.entity";
import { CreateUserDto } from "../dto/create-user.dto";

@Injectable()
export class UsersService {
  // ▼ TypeORM 방식: 엔티티마다 Repository<T>를 주입받는다.
  constructor(
    @InjectRepository(User)
    private readonly usersRepository: Repository<User>
  ) {}
}
```

- `@InjectRepository(User)` — User 엔티티 전용 리포지토리를 생성자로 주입한다. 모듈에서 `forFeature([User])` 로 등록해 뒀기에 NestJS가 이 자리에 알맞은 객체를 쫓는다.
- `private readonly usersRepository: Repository<User>` — 주입받은 리포지토리를 필드로 보관한다. 타입이 `Repository<User>` 라서, 이 객체의 메서드들은 모두 User를 다룬다는 게 타입으로 보장된다.

이제 CRUD를 메서드별로 보자.

```
create(dto: CreateUserDto): Promise<User> {
  const user = this.usersRepository.create(dto); // 엔티티 인스턴스 생성
  return this.usersRepository.save(user); // INSERT
}
```

생성은 두 단계다. `create(dto)` 는 DTO를 받아 메모리상의 엔티티 인스턴스를 만들 뿐, 아직 DB에 닿지 않는다.

`save(user)` 가 실제 INSERT 를 날린다. 반환 타입이 `Promise<User>` 인 데 주목한다. DB 작업은 네트워크 너머의 일이라 항상 비동기다. 10장이 바로 User 를 돌려줬다면, 여기서는 `Promise<User>` 다.

```
findAll(): Promise<User[]> {
  // 연관 데이터는 relations 옵션으로 명시해야 함께 로드된다.
  return this.usersRepository.find({ relations: ["posts"] });
}
```

`find()` 는 `SELECT * FROM user` 에 해당한다. 핵심은 `relations: ["posts"]` 다. 이걸 빼면 사용자 목록만 오고 각자의 글은 안 따라온다. 연관 데이터는 자동으로 떨어져오지 않는다. 필요할 때 명시적으로 불러오는 게 TypeORM의 기본값이다. `["posts"]` 를 주면 각 사용자에게 `posts` 배열이 채워져 응답에 함께 실린다.

```

async findOne(id: number): Promise<User> {
  const user = await this.usersRepository.findOne({
    where: { id },
    relations: ["posts"],
  });
  if (!user) throw new NotFoundException(`사용자 ${id}를 찾을 수 없습니다.`);
  return user;
}

```

단건 조회는 `findOne` 에 `where` 조건을 준다. `where: { id }` 가 `WHERE id = ?` 로 번역된다. 못 찾으면 `findOne` 은 `null` 을 돌려주는데, 그대로 두면 응답이 비어 클라이언트가 오해한다. 그래서 `NotFoundException` 을 던진다. 이 예외는 NestJS가 받아 자동으로 **404** 응답으로 바꾼다. 10장과 던지는 예외가 똑같다 — DB가 붙어도 "없으면 404"라는 규칙은 그대로다.

```

async remove(id: number): Promise<void> {
  const result = await this.usersRepository.delete(id);
  if (result.affected === 0) {
    throw new NotFoundException(`사용자 ${id}를 찾을 수 없습니다.`);
  }
}

```

삭제는 `delete(id)` 한 번이다. 10장처럼 배열에서 인덱스를 찾아 `splice` 할 필요가 없다. 다만 "정말 지웠는지"는 확인해야 한다. `delete` 가 돌려주는 `result.affected` 는 실제로 지워진 행의 수다. 없는 id를 지우라고 하면 0이 나오고, 그땐 404를 던진다. 배열 시절의 `findIndex === -1` 검사가 `affected === 0` 검사로 자리를 옮겼다.

연관 로드 — 작성자를 함께 불러오기

게시글 서비스는 사용자 서비스와 판박이다. 리포지토리만 `Repository<Post>` 로 바뀐다. 새로 볼 건 조회에서 작성자를 함께 불러오는 부분이다.

```

findAll(): Promise<Post[]> {
  return this.postsRepository.find({ relations: ["author"] });
}

async findOne(id: number): Promise<Post> {
  const post = await this.postsRepository.findOne({
    where: { id },
    relations: ["author"],
  });
  if (!post) throw new NotFoundException(`게시글 ${id}를 찾을 수 없습니다.`);
  return post;
}

```

`relations: ["author"]` 가 각 게시글에 작성자 객체를 채워 넣는다. 이게 없으면 응답에는 `authorId: 1` 같은 숫자만 있고 작성자의 이름·이메일은 빠진다. `["author"]` 를 주면 TypeORM이 `posts`와 `user`를 `JOIN` 해서, 각 글에 `author: { id, email, name, ... }` 객체를 통째로 붙인다. 10장에서는 `authorId` 숫자만 들고 있어 작성자 정보를 따로 조회해야 했지만, 여기서는 관계 한 줄로 끝난다.

`logging: true` 덕분에, 이 요청을 보내면 콘솔에서 실제 `LEFT JOIN` SQL을 확인할 수 있다. ORM이 관계를 어떻게 SQL로 푸는지 직접 보는 좋은 기회다.

생성·삭제는 사용자 서비스와 같은 패턴이다. `create` 는 `create()` 로 인스턴스를 만들고 `save()` 로 `INSERT` , `remove` 는 `delete(id)` 후 `affected` 를 확인한다.

HTTP 계층 — 안 바뀌는 입구

```
// 이 컨트롤러는 10장·12장의 컨트롤러와 글자까지 동일하다.
// "바뀌는 건 데이터 계층(서비스)뿐, HTTP 계층은 그대로"가 핵심 교훈.
@Controller("users")
export class UsersController {
  constructor(private readonly userService: UsersService) {}

  @Post()
  create(@Body() dto: CreateUserDto) {
    return this.userService.create(dto);
  }

  @Get()
  findAll() {
    return this.userService.findAll();
  }

  @Get(":id")
  findOne(@Param("id") id: string) {
    return this.userService.findOne(+id);
  }

  @Delete(":id")
  remove(@Param("id") id: string) {
    return this.userService.remove(+id);
  }
}
```

이 컨트롤러는 10장의 컨트롤러와 글자 하나 다르지 않다. 그게 이 장의 가장 큰 교훈이다. 저장소를 배열에서 PostgreSQL로 통째로 바꿨는데, HTTP 입구는 손댈 필요가 없었다. 컨트롤러는 "요청을 받아 서비스에 넘기고, 서비스가 돌려준 걸 응답한다"는 역할만 하고, 그 서비스가 안에서 배열을 뒤지든 DB에 질의하든 모른 채 지낸다.

- `@Param("id") id: string` → `this.usersService.findOne(+id)` — 경로 파라미터는 문자열이라 `+id` 로 숫자로 바꿔 넘긴다. 서비스의 `findOne(id: number)` 와 타입을 맞춘다.
- 서비스가 `Promise` 를 돌려줘도 컨트롤러는 `return` 만 하면 된다. NestJS가 `Promise`를 풀어 응답으로 내보낸다. `async / await` 를 컨트롤러에 따로 쓸 필요가 없다.

요청 검증용 DTO도 10장과 같다. **DTO와 컨트롤러는 ORM과 무관한 계층**이라, 데이터 저장 방식이 바뀌어도 그대로 간다. 저장소를 바꿔도 위 계층이 흔들리지 않는다는 점, 이것이 계층 분리의 값어치다.

10장 인메모리 vs 11장 TypeORM

	10장 (인메모리)	11장 (TypeORM)
저장소	<code>private users: User[] = []</code>	PostgreSQL + <code>Repository<User></code>
모델 정의	<code>interface User</code> (타입만)	<code>@Entity</code> 클래스(테이블 스키마)
기본키	<code>this.nextId++</code> 수동	<code>@PrimaryGeneratedColumn()</code> 자동
생성 시각	<code>new Date()</code> 수동	<code>@CreateDateColumn()</code> 자동
중복 이메일	<code>users.some()</code> 직접 검사	<code>@Column({ unique: true })</code> DB 제약
단건 조회	<code>users.find(u => u.id === id)</code>	<code>repo.findOne({ where: { id } })</code>
삭제	<code>splice(index, 1)</code>	<code>repo.delete(id)</code> + <code>affected</code> 확인
관계	<code>authorId</code> 숫자만 보관	<code>@ManyToOne</code> + <code>relations: ["author"]</code> 로 객체 로드
반환 타입	<code>User</code> (동기)	<code>Promise<User></code> (비동기)
컨트롤러·DTO	—	그대로(HTTP 계층은 불변)
데이터 영속성	서버 끄면 소멸	디스크에 영구 저장

마이그레이션은 왜 여기서 안 다루나

운영 환경에서는 `synchronize` 를 끈다. 대신 스키마 변경을 **마이그레이션**으로 관리한다. 마이그레이션은 "이번 배포에서 테이블을 이렇게 바꾼다"는 변경 한 단계를 파일로 남기는 방식이다. 무엇이 언제 어떻게 바뀌었는지 기록이 남고, 잘못되면 되돌릴 수 있다. 변경을 자동으로 맞추는 `synchronize` 와 달리, 사람이 변경을 통제한다.

이 단계에서는 마이그레이션을 다루지 않는다. `synchronize: true` 로 대체했기 때문이다. TypeORM의 마이그레이션 CLI를 돌리려면 별도의 `DataSource` 설정 파일이 필요한데, 학습 단계에서는 그 설정을 두지 않는다. 아래 명령은 참고용이고, 설정 파일(`-d` 가 가리키는 `DataSource`)이 갖춰져야 동작한다.

```
# (DataSource 설정 파일이 별도로 필요)
npx typeorm migration:generate ./src/migrations/Init -d ./src/data-source.ts
npx typeorm migration:run -d ./src/data-source.ts
```

정리하면, 여기서는 학습 편의를 위해 `synchronize` 로 스키마를 자동 동기화하고, 마이그레이션은 의도적으로 생략한다. 실무로 가면 이 자리를 마이그레이션이 채운다. 12장의 Prisma는 마이그레이션이 CLI에 기본으로 들어 있어, `npx prisma migrate dev` 한 줄로 끝난다. 같은 일을 두 ORM이 얼마나 다르게 다루는지 비교해 보면 좋다.

흔한 실수

- `.env` 를 안 채우고 실행한다. → DB 접속 실패. `cp .env.example .env` 후 `DB_USER` / `DB_PASSWORD` 를 본인 계정으로 바꿔야 한다. 비밀번호가 없는 계정이면 `DB_PASSWORD` 를 빈 값으로 둔다.
- DB를 안 만들고 실행한다. → `database "nest_typeorm" does not exist`. 테이블은 앱이 만들지만, 빈 DB는 먼저 만들어 줘야 한다(`createdb nest_typeorm`). 자동 생성되는 건 테이블이지 데이터베이스가 아니다.
- 모듈에 `forFeature` 를 빠뜨린다. → 부팅 시 "리포지토리를 주입할 수 없다"는 에러. 엔티티를 쓰는 모듈마다 `TypeOrmModule.forFeature([Entity])` 를 import해야 한다.
- `relations` 를 안 줘서 연관 데이터가 안 온다. → 사용자에게 `posts` 가 없거나, 게시글에 `author` 가 빠진다. 연관 데이터는 자동으로 안 떨어오니 `relations: [.]` 로 명시한다.
- `synchronize: true` 를 운영에 켜 둔다. → 엔티티를 고치는 순간 운영 테이블이 바뀌고, 열을 지우면 데이터가 날아간다. 운영에서는 반드시 끄고 마이그레이션을 쓴다.
- `import "dotenv/config"` 를 아래쪽에 둔다. → DB 설정이 `process.env` 를 읽을 때 값이 비어 있어 기본값으로 엉뚱한 계정에 붙는다. 이 import는 항상 맨 위다.

직접 해보기

1. 게시글 서비스에 `update` 메서드를 추가해 보자. 컨트롤러에 `@Put(":id")` 를 달고, 서비스에서 `findOne` 으로 글을 찾은 뒤 제목·내용을 바꾸고 `save` 하면 된다. 10장에서 했던 부분 수정(`undefined`)을 떠올린다.
2. 같은 이메일로 사용자를 두 번 생성해 보자. `@Column({ unique: true })` 때문에 DB가 거부하며 에러를 던진다. 이 에러를 잡아 10장처럼 `ConflictException` (409)으로 바꿔 응답해 보자.
3. 사용자를 하나 만들고 그 사람 이름으로 글을 몇 개 쓴 뒤, 그 사용자를 `DELETE` 해 보자. `onDelete: "CASCADE"` 덕분에 글까지 함께 사라지는지 `GET /posts` 로 확인한다.

정리

- ORM은 테이블을 클래스로, 행을 인스턴스로, SQL을 메서드로 바꾼다. TypeORM의 `@Entity` 클래스가 곧 테이블 스키마다.
- DB 연결은 루트의 `forRoot` 에서 한 번, 모듈별 리포지토리는 `forFeature` 로 등록한다. 서비스는 `@InjectRepository` 로 `Repository<T>` 를 주입받아 CRUD를 처리한다.
- 관계는 `@ManyToOne` / `@OneToMany` 로 맺고, 연관 데이터는 `relations` 옵션으로 명시해 불러온다. 자동으로 딸려 오지 않는다.
- 저장소를 배열에서 DB로 바뀌도 **컨트롤러와 DTO는 그대로였다**. 계층 분리 덕에 변화가 서비스 한 곳에 갇힌다.
- `synchronize: true` 는 학습용 편의이고, 운영에서는 끄고 마이그레이션으로 스키마를 통제한다. 이 앱은 마이그레이션을 생략했다.

다음 장(12 · NestJS + Prisma)에서는 똑같은 도메인을 또 다른 ORM인 Prisma로 만들며, 스키마 정의와 마이그레이션이 TypeORM과 어떻게 다른지 비교한다.